SmartDigiCat

User manual

Version 1.0

From a table of measurements
to a predictive model and optimal conditions

Author: Komlan NOUKPOAPE

Industrial chair – Centrale Lille

26 September 2026

Contents

Foreword **2**

1 SmartDigiCat, an executable application **2**
1.1 What “executable” means . . . 2
1.2 Starting the application . . . 3
1.3 Window layout . . . 3
1.4 The project concept . . . 3
1.5 Language, font and keyboard . . . 4
1.6 Supported data formats . . . 4

2 Button functions **4**
2.1 Top bar . . . 4
2.2 Left side bar . . . 5
2.3 Panel buttons . . . 6
2.4 Common check boxes . . . 7

3 Statistical analyses **7**
3.1 Preparing the data (Edit data) . . . 7
3.2 Table overview (Overview stat) . . . 7
3.3 Descriptive statistics and correlations (Continuous columns) . . . 8
3.4 Comparing groups (All columns) . . . 8
3.5 Correlation with the target (Plot corr) . . . 9
3.6 Group summaries (Other analysis) . . . 9
3.7 Exploration charts . . . 9
3.8 Exporting the results . . . 10

4 Regression models: from creation to prediction **10**
4.1 Available algorithms . . . 10
4.2 Workflow overview . . . 10
4.3 Step 1: prepare the project and the data . . . 11
4.4 Step 2: create a model box . . . 11
4.5 Step 3: configure the model . . . 11
4.6 Step 4: train with Run . . . 12
4.7 Step 5: assess the model . . . 12
4.8 Step 6: predict new experiments . . . 13

5 Optimisation **13**
5.1 Principle of Bayesian optimisation . . . 13
5.2 Opening the panel . . . 14
5.3 Defining the problem (Optimization settings) . . . 15
5.4 Running and following the optimisation . . . 15
5.5 Reading the result . . . 15
5.6 Exporting and using the points . . . 16

Foreword

SmartDigiCat lets a chemist or an engineer go, without writing any code, from a table of experimental measurements to a predictive model, and then to optimised operating conditions. This manual follows that path in the order in which the user takes it.

Intended readers. The manual is written for master's students, PhD students and researchers in catalysis, process engineering or data science. It assumes a working knowledge of descriptive statistics. Machine-learning and Bayesian-optimisation concepts are explained where they first appear.

Structure. The manual has five sections, matching the main stages of a study.

Section	Content	Question it answers
1	The executable software	How do I start SmartDigiCat and find my way around?
2	The buttons	What does each button do?
3	Statistical analyses	What do my data say before any modelling?
4	Regression models	How do I build, assess and use a model?
5	Optimisation	Which conditions maximise my response?

Reading conventions.

- Interface labels are shown in bold, exactly as on screen: **Run**, **Edit data**, **Save configure**.
- A sequence of clicks is written with an arrow: **Plots** → **Statistical data** → **View stat**.
- The *target* (or response variable) is the quantity to predict, for example a yield. The *features* (or explanatory variables) are the quantities used to predict it, for example temperature or pressure.
- The "Teaching note" boxes explain the statistical meaning of a result, not only how to obtain it.

1. SmartDigiCat, an executable application

SmartDigiCat v1.0 is a standalone desktop application: it starts with a double-click, without installing Python or any library. It comes from the SmartDigiCat scientific project, which brings together catalysis, data science and artificial intelligence, led by the industrial chair of Centrale Lille.

1.1 What "executable" means

The program is written in Python and then "frozen" with the PyInstaller tool. Freezing bundles the Python interpreter, the scientific libraries and the graphical resources into a single package. The user therefore receives a ready-to-use program.

Bundled component	Role in SmartDigiCat
Tkinter and tkmacosx	Graphical interface: windows, buttons, lists
pandas and NumPy	Reading, filtering and computing on data tables
scikit-learn and mlxtend	Regression models, cross-validation, feature selection
SciPy	Statistical tests and probability distributions for optimisation
Matplotlib	All charts
python-docx and openpyxl	Word and Excel exports
`figures` folder	Button icons

Teaching note

Freezing a program guarantees reproducibility: two users with the same version get the same results, because they use exactly the same library versions.

1.2 Starting the application

1. Copy the application folder to the disk, keeping the executable together with its resources.
2. Double-click the SmartDigiCat executable. On macOS, the first launch may require permission in the Security settings.
3. The main window opens; its title bar shows “SmartDigiCat v1.0”, followed by the name of the open project.

Long computations run in the background: the interface stays responsive and a stop button can interrupt them.

1.3 Window layout

The main window has four fixed areas.

Area	Position	Content
Top bar	Top	Project, data, model and result management
Side bar	Left	Model actions, stop, capture, export, language, zoom
Central panel	Centre	Workspace: tables, charts, settings
Bottom bar	Bottom	Status messages (undo, redo)

Each top-bar button shows its own panel in the central panel. The vertical slider in the left bar sets the zoom of the displayed chart.

1.4 The project concept

Every study takes place in a *project*, saved in a JSON file. The project keeps:

- the path of the data file and the filters applied;
- all models, with their configuration, results, colour and position;
- the optimisation settings and the latest optimisation result;
- the chart display settings.

Three mechanisms protect the work:

- **Autosave**: the project is saved every 5 minutes.
- **History**: the last 20 actions can be undone (**Undo**) or redone (**Redo**).
- **Protected closing**: on closing, a dialog offers to save, quit without saving, or cancel.

1.5 Language, font and keyboard

The language button in the left bar selects the language of the instructions, as well as the font and its size. The Enter key confirms every dialog box and filtering window; the Escape key closes the dialogs that allow it.

1.6 Supported data formats

The data table is imported as CSV, Excel (`.xlsx`) or text (`.txt`). It must contain one row per experiment and one column per quantity. The first row holds the column names. Columns can be quantitative (numbers) or qualitative (text, for example a catalyst name).

2. Button functions

Buttons are grouped into two permanent bars and into buttons specific to each panel. Buttons that need an open project stay greyed out until a project is open.

2.1 Top bar



Figure 1: SmartDigiCat top bar, from left to right.

Button	Function	When to use it
New	Creates a project: name, then location of the JSON file	At the start of every study
Open	Opens an existing JSON project and restores data, models and results	To resume a study
Save	Saves the open project	After an important step
Undo	Undoes the last action (20 levels)	After a handling mistake
Redo	Redoes the undone action	After one **Undo** too many
Import data	Imports the data table (CSV, XLSX, TXT)	Right after creating the project
Edit data	Opens the data preparation panel (section 3.1)	Before any analysis
Plots	Opens the graphical and statistical analyses, and optimisation	Exploration (section 3) and optimisation (section 5)
Model	Shows the project's model boxes	Creation and configuration (section 4)
Run	Trains all configured models	After configuration
Log	Opens the prediction panel and the model charts	After training
Help	Shows this manual in the central panel, in the chosen language	To look up an explanation
Website	Opens the SmartDigiCat project web page	For the scientific background

2.2 Left side bar

The first five buttons act on the model selected in the **Model** panel.

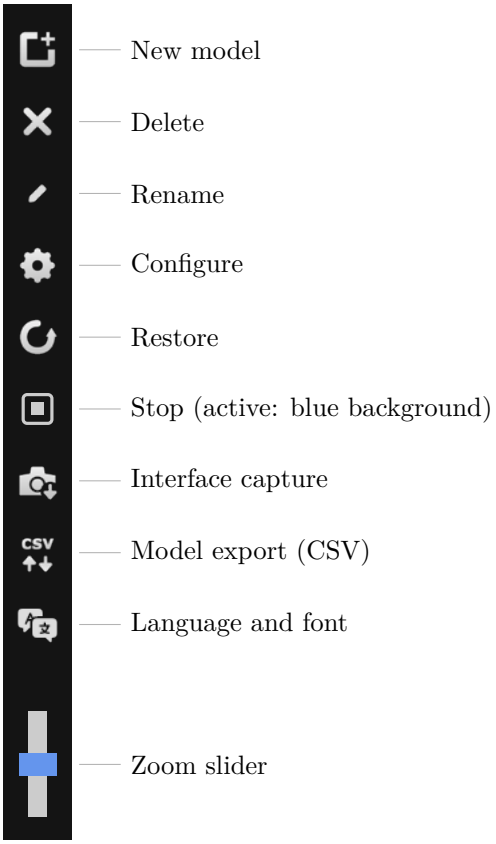


Figure 2: Left side bar. The light icons are shown on a dark background, as in the software.

Button	Function
New model	Creates a model box: name, then optional colour
Delete	Moves the selected model to the recycle bin
Rename	Changes the name and colour of the box
Configure	Opens the algorithm choice and its settings
Restore	Opens the recycle bin: restore or delete permanently
Stop	Interrupts the training or optimisation in progress; active (blue background) only during a run
Capture	Saves a PNG image of the interface
Model export	Exports the list of models and their results as CSV
Language and font	Selects the language of the instructions, the font and its size
Vertical slider	Zoom of the displayed chart

Teaching note

During a run, every button except *Stop* is locked and an "Execution in progress…" box appears. This lock prevents the data from being changed while a model is using them. The box can be moved with the mouse.

2.3 Panel buttons

Panels reuse the same buttons, always with the same meaning.

Button	Panels	Function
View plot	Charts, results, optimisation	Draws or redraws the chart
Save plot	Charts, results	Saves the chart as PNG, PDF or JPEG
View stat	Statistical data	Computes and shows the statistical tables
Export stat	Edit data, Statistical data	Exports the tables to Excel, Word or CSV
Overview stat	Edit data, Log	Summary table of all columns
Export clean	Edit data	Exports the cleaned data
View & Edit	Edit data	Shows the table and allows editing
Save	Filters, recoding, settings	Confirms the window; the Enter key does the same
Save configure	Model configuration	Stores the configuration for the next **Run**
Import data	Log, optimisation	Imports a prediction or optimisation table
Show all	Log	Shows the prediction data
Export pred	Log	Exports the predictions
Run optim	Optimisation	Starts the optimisation
Optimization settings	Optimisation	Opens the optimisation settings

2.4 Common check boxes

Check box	Effect
Define chart title	Opens a window for the title and axis names
Detail of model	Shows model details: settings, R^2, MSE, number of observations
Model plots (Log)	Shows the charts of all trained models over the whole panel
Train data	Adds the training points to the results chart

3. Statistical analyses

Before modelling, SmartDigiCat lets you prepare the data, describe them, measure the links between variables and visualise them. These analyses are descriptive: they summarise the sample and guide the choice of features, without predicting anything yet.

3.1 Preparing the data (Edit data)

Poor-quality data produce poor-quality models. The **Edit data** panel gathers the cleaning operations.



Figure 3: Buttons of the **Edit data** panel.

Operation	Settings	Effect
Inspection (**View & Edit**)	Columns and number of rows to show	Shows the table and allows corrections
Search and replace a category	Column, **Search**, **Replace**	Fixes a mistyped category (e.g. “Pt/Al2O3” and “Pt-Al2O3”)
Renaming (**Rename column**)	Old and new name	Clarifies a column name
Missing values, quantitative	mean, median or delete	Replaces with the mean or median, or deletes the row
Missing values, qualitative	modality reference or delete	Replaces with the most frequent category, or deletes the row
Filtering (**Parameter 1** to **4**)	Column, Min, Max	Keeps only the rows within the chosen range
Recoding (**Categorizing variables**)	Columns to categorise	Turns a numerical variable into classes

Teaching note

The median is more robust than the mean to extreme values. For a skewed variable, such as a conversion often close to 0 or 100 %, median imputation is preferable. Deleting incomplete rows is the safest choice when there are few of them.

3.2 Table overview (Overview stat)

Edit data → **Overview stat** produces an identity card for each column.

Table column	Meaning
Characteristics	Variable name
Type	double, integer (quantitative) or character (qualitative)
Unique	Number of categories of a qualitative variable
Modality	Name of each category
Missing	Number of missing values
N	Number of available values
mean (SD) / (%)	Mean (standard deviation) for a quantitative variable; percentage for a category

The title recalls the table size (rows, columns). Qualitative variables are highlighted, with their categories listed below.

3.3 Descriptive statistics and correlations (Continuous columns)

Plots → **Statistical data**, choose **Continuous columns**, then **View stat**. Two tables appear for all numerical columns.



Figure 4: Buttons of the **Statistical data** panel (sections 3.3 to 3.6).

Descriptive statistics table. It gives N, Missing, Mean, Std, Min, Q1, Median, Q3 and Max. Q1 and Q3 are the first and third quartiles: 50 % of the observations lie between them.

Pearson correlation table. Each cell holds the coefficient r between two variables:

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\,\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{1}$$

Value of r	Interpretation
Close to $+1$	Both variables increase together, linearly
Close to 0	No linear link (a non-linear link is still possible)
Close to -1	When one increases, the other decreases

Teaching note

Correlation is not causation. Two strongly correlated features ($|r| > 0.9$) carry almost the same information: keeping only one often stabilises linear models.

3.4 Comparing groups (All columns)

Choose **All columns**, then a qualitative column in **Groupby** (for example the catalyst). The table describes each variable within each group and tests whether the groups differ.

Variable type	Summary per group	Statistical test	Null hypothesis H_0
Quantitative	median (Q1; Q3)	Kruskal–Wallis	The groups have the same distribution
Qualitative	count N (%) per category	χ^2 test of independence	The variable is independent of the group

The Kruskal–Wallis statistic compares the mean ranks of the groups, without assuming a normal distribution:

$$H = \frac{12}{N(N+1)} \sum_{g=1}^{k} \frac{R_g^2}{n_g} - 3(N+1) \tag{2}$$

Here N is the total sample size, k the number of groups, n_g and R_g the size and rank sum of group g.

Reading the p-value. A p-value below 0.05 leads to rejecting H_0 at the 5 % level: the variable differs significantly between groups. The software shows "<0.001" for very small values.

3.5 Correlation with the target (Plot corr)

In **Statistical data**, choose a **Target** and **Features**, then **Plot corr**. A bar chart shows the Pearson coefficient between the target and each feature. Positive and negative bars have different colours. This chart helps pre-select the features of a model.

3.6 Group summaries (Other analysis)

Choose a **Dimension** (grouping column), a **Method** (min, mean, max or median) and parameters. The software computes the requested statistic for each group and draws a grouped bar chart. Example: the mean yield per catalyst.

3.7 Exploration charts



Figure 5: Menu of the **Plots** button: charts, statistics and optimisation.

Chart (Plots)	What it shows	Typical use
Scatter plot	X–Y point cloud, colour and size from other columns	Spot a trend or outliers
Box plot	Median, quartiles and extreme values per group	Compare distributions
Histogram	Distribution of a variable's values	Check the shape of a distribution
plot 3D	Point cloud in X–Y–Z space	See the effect of two features on the target
Data table	Data table	Visual check

Every chart accepts a title and axis names (**Define chart title**), can be zoomed with the left-bar slider and exported with **Save plot**.

3.8 Exporting the results

Export stat exports all displayed tables at once: to Excel (one sheet per table, formatting kept), to Word (landscape document) or to CSV.

4. Regression models: from creation to prediction

SmartDigiCat offers eight regression algorithms. A model learns a function f such that $y \approx f(\mathbf{x})$, where y is the target and $\mathbf{x}$ the feature vector. It is assessed on data it has not seen, then used to predict new experiments.

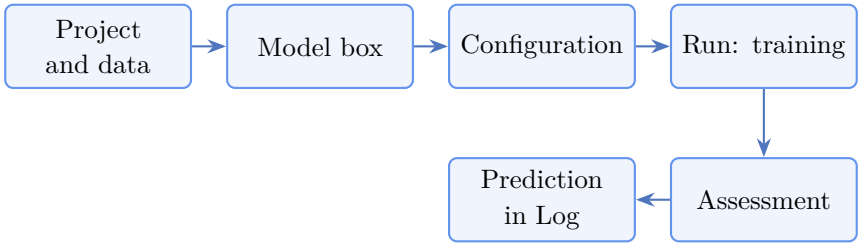
4.1 Available algorithms

Algorithm	Principle	Tunable settings (default)	Strengths
Linear	Least-squares line (or hyperplane)	None	Simple, interpretable coefficients
Lasso	Linear regression penalised by the sum of |coefficients|	alpha (1.0)	Sets useless features to zero
Kernel ridge (KR)	Ridge regression in a kernel space	alpha (1.0)	Stable with correlated features
Support Vector Regression (SVR)	Tolerance tube around the function, Gaussian (RBF) kernel	C (100), gamma (0.1)	Non-linear relations, few experiments
Random Forest (RF)	Average of many trees built on random samples	number of trees (100), random state (0)	Robust, feature importance
Generalized Boosted Regression Modeling (GBM)	Trees added one by one, each correcting the previous errors	number of trees (500), depth (5), learning rate (0.5), loss function	Very accurate on complex relations
Decision Tree (DT)	Successive splits of the data at thresholds	max depth (20), random state (0)	Readable as rules
Gaussian process (GP)	Probability distribution over functions, dot-product + white-noise kernel	random state (0)	Gives an uncertainty on the prediction

Teaching note

A model that is too simple underfits: it misses the trend. A model that is too flexible overfits: it reproduces the noise of the training data and predicts new data poorly. The settings alpha, C, depth or number of trees control this balance.

4.2 Workflow overview



Each step is detailed below. Several models can follow this path in parallel within the same project, so that they can be compared.

4.3 Step 1: prepare the project and the data

1. **New**: create the project.
2. **Import data**: load the table.
3. **Edit data**: clean, filter and impute missing values (section 3.1).

4.4 Step 2: create a model box

1. Click **Model**, then the *New model* button in the left bar.
2. Enter a name (for example "RF temperature") and, if you wish, a colour.
3. A box appears in the panel. It can be moved with the mouse.

The box shows the model name and its status: configured, then trained. Once the model is configured, the chosen algorithm appears at the top of the box (its acronym, for example "GBM", if the name is too long).

4.5 Step 3: configure the model

Select the box, then click *Configure* (or double-click the box). Choose the algorithm from the list, then fill in the fields.

Field	Role	Advice
Target column	Variable to predict	A single numerical column
Calculation method	TrainTestSplit data or Cross-validation	See section 4.6
Test size / Split size	Test fraction (TrainTestSplit) or number of folds K (Cross-validation)	0.2 and $K = 5$ to 10 are common choices
Feature columns	Candidate explanatory variables	Exclude the target and identifiers
Recoding method	None, Disjunctive or All-disjunctive	See below
Recoding feature columns	Columns to recode (Disjunctive mode)	The qualitative columns
Algorithm settings	alpha, C, gamma, depth…	Keep the defaults for a first try

Recoding qualitative variables. An algorithm only handles numbers. A qualitative column with m categories is therefore turned into $m - 1$ 0/1 indicator variables. The most frequent category is the reference and has no indicator. Qualitative columns are always recoded, even in None mode.

Finally, click **Save configure**. The configuration is stored and will be run at every **Run**, without reopening this panel.



Figure 6: Buttons of a model's configuration and results.

4.6 Step 4: train with Run

Run trains all configured models of the project. The **Model** panel stays displayed, with the progress box. The *Stop* button interrupts the run: the models already trained keep their results.

In both methods, the data are first split into a training part and a test part, with a fixed seed so that the result is reproducible.

Method	Training	Test	What is optimised
TrainTestSplit data	1− Test size (80 % by default)	Test size (20 %)	Feature selection
Cross-validation	80 %	20 %	Hyperparameters by K-fold cross-validation, then feature selection

K-fold cross-validation. The training part is split into K blocks. The model is trained K times, each time on $K-1$ blocks, and assessed on the remaining block. The hyperparameters kept minimise the mean error over these K trials. The 20 % test part is used only for the final assessment.

Feature selection. Sequential forward selection starts from a model with no feature. At each step it adds the feature that improves the score most, then keeps the best subset.

4.7 Step 5: assess the model

Two indicators are computed on the test data, where y_i is the measured value and $\hat{y}_i$ the predicted value:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3}$$

Indicator	Interpretation
R^2 close to 1	The model explains almost all the variability of the target
R^2 close to 0	No better than predicting the mean
Negative R^2	Worse than the mean: the model must be revised
MSE	Mean squared error, in squared target units; the lower, the better

Results are shown in **Log**, check box **Model plots**: one tab per model.

- **Predicted vs measured chart**: one point per test experiment, measured value on the x-axis and predicted value on the y-axis. The closer the points are to the diagonal, the better the model. The **Train data** check box adds the training points.
- **Detail of model**: algorithm, settings, target, best features kept, coefficients (Linear, Lasso) or importances (RF, GBM, DT), R^2, MSE and number of training and test observations.

Teaching note

A training R^2 much higher than the test R^2 signals overfitting. Showing the training points with **Train data** makes it directly visible.

4.8 Step 6: predict new experiments

The **Log** panel automatically uses the best trained model, the one with the highest test R^2.



Figure 7: Buttons of the **Log** panel (prediction).

1. **Import data**: load a table containing the same features as the model.
2. If needed, filter (Parameter 1 to 4) and impute missing values, as in **Edit data**.
3. **Show all**: show the data with the added prediction column.
4. **Overview stat** and **Detail of model**: check the data and recall the model used.
5. **Export pred**: export the table with the predictions.

Qualitative columns are recoded exactly as during training. If a model feature is missing, the prediction is refused with an explicit message.

Teaching note

A model only predicts reliably within the domain covered by its training data. Predicting at a temperature well above the highest measured temperature is an extrapolation, to be interpreted with caution.

5. Optimisation

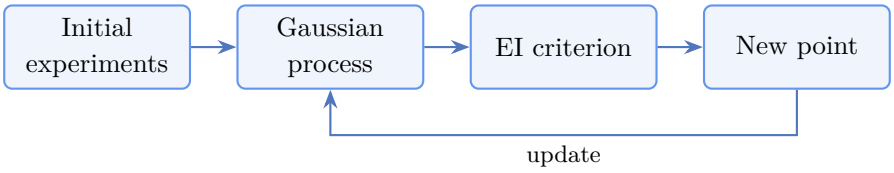
Optimisation looks for the feature values that maximise the target, for example the operating conditions that give the best yield. SmartDigiCat uses Bayesian optimisation, suited to situations where each experiment is expensive and few points are available.

5.1 Principle of Bayesian optimisation

The idea is to alternate two actions:

1. **Model** the unknown response $f(\mathbf{x})$ with a Gaussian process, built from the experiments already carried out. This model gives a prediction and an uncertainty at every point.
2. **Choose** the next point that offers the best trade-off between exploitation (where the prediction is high) and exploration (where the uncertainty is large).

The new point is added to the data, the model is updated, and the cycle starts again.



The Gaussian process. The values of f at n points are assumed to follow a multivariate normal distribution. The correlation between two points decreases with their distance, following a Matérn kernel:

$$k_\nu(d) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu}\, d\right)^{\nu} K_\nu\left(\sqrt{2\nu}\, d\right), \qquad d = \sqrt{\sum_{j=1}^{p} \left(\frac{x_j - x'_j}{\theta_j}\right)^2} \tag{4}$$

Parameter	Meaning	Estimation in SmartDigiCat
θ_j (range)	Distance over which feature j influences the response	Sampled by Markov chain Monte Carlo
ν (smoothness)	Degree of smoothness of the function	Sampled by Markov chain Monte Carlo
σ^2 (variance)	Amplitude of the response variations	Posterior distribution computed analytically

K_ν is the modified Bessel function of the second kind. The parameters are not fixed once and for all: they are sampled from their posterior distribution with a Metropolis–Hastings algorithm. This fully Bayesian approach carries the uncertainty on the parameters through to the choice of the next point.

The expected improvement (EI) criterion. Let f^* be the best observed value, and $\mu(\mathbf{x})$ and $s(\mathbf{x})$ the predicted mean and standard deviation at $\mathbf{x}$. The criterion is:

$$\mathrm{EI}(\mathbf{x}) = (\mu(\mathbf{x}) - f^*)\,\Phi(z) + s(\mathbf{x})\,\varphi(z), \qquad z = \frac{\mu(\mathbf{x}) - f^*}{s(\mathbf{x})} \tag{5}$$

Φ and φ are the cumulative distribution function and the density of the standard normal distribution. The first term rewards a high prediction, the second a large uncertainty. The proposed point is the one that maximises EI.

Teaching note

Looking only for the maximum of the prediction would trap the algorithm around the best known point. The uncertainty term also pushes it towards little-explored regions, where a better optimum may lie.

5.2 Opening the panel

Plots → Optimization. The left area shows the chart; the right area contains three groups of settings: Imput, Output and Export.



Figure 8: Buttons of the optimisation panel.

5.3 Defining the problem (Optimization settings)

Field	Role	Example
Optimization method	FBA, EGO quanti or EGO mixt	FBA
Target column	Quantity to maximise	Yield
Target Min / Max	Plausible range of the target	0 and 100 (%)
Feature columns	Features to vary	Temperature, pressure
Feature Min / Max	Search domain of each feature, comma-separated	200,1 and 400,10
N	Number of iterations, i.e. of proposed points	10
seed	Random-number generator seed	0

The following checks are made before computing: Min $<$ Max for the target and each feature, $N \geq 2$, and one Min and one Max per feature. A message shows when all settings are valid; then click **Save**.

In version 1.0, choosing a method is mandatory, but the three options run the same Bayesian algorithm described above.

Teaching note

The seed makes the result reproducible: the same data, settings and seed give the same points. Changing the seed checks that the proposed points do not depend on chance.

5.4 Running and following the optimisation

1. If needed, **Import data** (Imput group) loads a dedicated table; otherwise the project data are used.
2. Click **Run optim**. The chart area is cleared and the progress box appears in it.
3. At the end, the panel is refreshed and the chart appears. The *Stop* button interrupts the computation; the previous result is then kept.

The N points are proposed one after another: each one takes all the previous ones into account.

5.5 Reading the result

The check boxes of the Output group build the chart.

Check box	Element drawn
Given data points	Initial experiments (black)
Optimized points	Points proposed by the optimisation (red)
Curve estimated	Response estimated by a Matérn Gaussian process (single feature)
Error (95%)	95 % confidence band around the curve
Optimized table	Table of the proposed points; **with given points** adds the initial experiments

Line width, **Point size** and the zoom slider set the appearance. The **X axis** list selects the feature on the x-axis; with several features, the chart is a projection onto that feature.

Teaching note

The 95 % band is narrow near the experiments and widens away from them. A point proposed in a wide area is an exploration point: it teaches the model a lot, even if it does not necessarily give the best yield.

5.6 Exporting and using the points

- **Save plot** saves the chart (PNG, PDF or JPEG).
- **Save points** exports the table of points to Excel or CSV.

The target values attached to the proposed points are model estimates, not measurements. The complete approach is to run these experiments in the laboratory, add the measured results to the table, then run the optimisation again. Each cycle refines the knowledge of the response around the optimum.